

Lecture 5 - Object Design: Case Study Presentations in Week 6

Topic: Responsibility-Driven Design (RDD), CRC Cards, Cohesion, Coupling, and the Towers of Hanoi

Format: Each Group: 5-10 Minute Live Presentation & Discussion

Architectural Strike Teams: The Towers of Hanoi Project

Background Context for All Groups:

"In the jungle outside Hanoi, monks are moving 64 rings stacked on 3 poles. A ring may only be moved one at a time, and a larger ring cannot be placed on top of a smaller ring. When all rings are moved from Pole 1 to Pole 3, the world will end."

Your software firm has been hired to build an object-oriented simulation engine for this scenario. Break into your assigned "Strike Teams" below to design the architecture.

Group 1: The Object Discovery Team

Focus: Noun-Phrase Analysis & Identifying Candidate Classes (*Refers to Tutorial Q1*)

Scenario: You are starting with a blank whiteboard. Before writing any CRC cards, you must define the "Cast of Characters" (the Classes) that will make up this biological community of objects.

Presentation Prompts:

1. **Noun Analysis:** Perform a quick noun-phrase analysis on the problem description. List the legitimate "Candidate Classes" you will build in code.
2. **Discard Pile:** Which nouns from the text did you aggressively discard and why? (Give an example of an external actor, an irrelevant concept, and a simple attribute).
3. **Roles:** Pick one of your core classes (e.g., the Pole). What is its Role? Is it a Controller, a Coordinator, or an Information Holder?

Group 2: The Responsibility Architects

Focus: Doing vs. Knowing & The Information Expert Principle (*Refers to Tutorial Q2 & Q3*)

Scenario: You have your blank CRC cards for your core classes. Now you need to assign responsibilities (obligations) without creating a centralized "God Class."

Presentation Prompts:

1. **Doing vs. Knowing:** Identify two "Knowing" responsibilities and two "Doing" responsibilities in this system. Which specific classes get assigned these responsibilities?
2. **The Information Expert:** A core rule of Hanoi is "a larger ring cannot be placed on a smaller ring." Applying the Information Expert principle, which class should be responsible for validating if a move is legal, and why?

3. **The Setup Problem:** The tutorial hints that you need a new class responsible for setting up the poles and starting the game. Why is it bad Object-Oriented Design to force the Ring or Pole class to build the whole game themselves?

Group 3: The Message-Passing Role-Players

Focus: Collaborations & "Tell, Don't Ask" (*Refers to Tutorial Q4*)

Scenario: Alan Kay said OOP is all about biological cells passing messages. You need to map out the exact sequence of messages (collaborations) required to successfully move a ring.

Presentation Prompts:

1. **Client-Server Contract:** When moving a ring from Pole 1 to Pole 3, define the Client-Server relationship. Who is the Client making the request, and who is the Server providing the service?
2. **The Walkthrough:** Trace the exact sequence of messages passed between objects to successfully move the top ring from Pole 1 to Pole 3.
3. **Tell, Don't Ask:** How do you move the ring *without* violating encapsulation? Explain how "Tell, Don't Ask" prevents you from just extracting the ring's size, checking the pole's array yourself, and forcing the data in.

Group 4: The Refactoring Analysts

Focus: Cohesion, Coupling, & Anti-Patterns (*Refers to Tutorial Q5*)

Scenario: A junior developer handed you their Hanoi design. They created a single Monk class that holds a giant 2D array of all rings and contains a massive 500-line calculateMove() method. Their Ring and Pole classes contain absolutely no methods, only variables with getters and setters.

Presentation Prompts:

1. **Evaluating Metrics:** Evaluate this junior developer's design using the concepts of *Cohesion* and *Coupling*. Is the cohesion high or low? Is the coupling tight or loose?
2. **Identifying Anti-Patterns:** What two specific OO anti-patterns has the junior developer created by designing the system this way? (Hint: Think about "God Classes" and "Anemic Domain Models").
3. **The Fix:** How do you redesign this to achieve "evenly distributed system intelligence" across the CRC cards?